

Substitution Classes as an Attention Mechanism: A Dynamic Alternative to Learned Embeddings

Claude Opus 4.5 (claude-opus-4-5-20251101)
prompted by Rob Freeman

January 28, 2026

Abstract

We describe a sequence prediction system that implements an attention-like mechanism through *substitution class expansion* rather than learned attention weights. The key insight is that units (words or phrases) which share context patterns form natural equivalence classes, and predictions can flow from all members of these classes. This achieves the same goal as transformer attention—relating inputs to contextually equivalent forms—but computes the relationships dynamically from corpus statistics rather than learned parameters. We detail the algorithm, recent GPU optimizations achieving 17-20x speedups, and the implementation of Lin’s information-theoretic similarity measure for improved structural filtering.

1 Introduction: The Attention Analogy

Transformer attention mechanisms work by learning which input positions should contribute to predicting each output position. The attention weights, computed as $\text{softmax}(QK^T/\sqrt{d})$, determine how much each input token “attends to” other tokens.

We propose that **substitution class membership** achieves the same goal through a different mechanism:

Transformer Attention	Substitution Classes
Learned weights determine which positions contribute	Class membership determines which expressions contribute
Query-key similarity selects relevant context	Context overlap defines substitutability
Values from attended positions are aggregated	Predictions from class members are aggregated
Static embeddings encode token relationships	Dynamic context signatures encode relationships

1.1 The Core Insight

Consider the phrase “the rain in spain”:

- The word “rain” has a *substitution class*—other expressions that appear in similar contexts
- The full phrase “the rain in spain” can also be a member of that class (they share context patterns)
- When predicting what follows “the rain in spain”, predictions come from the *entire substitution class*

- So “the rain in spain” inherits predictions from “rain”, “the weather”, “it”, etc.

This is attention: the input “attends to” its abstract equivalents, and their predictions contribute proportionally to their centrality in the class.

2 Algorithm Overview

2.1 Unit Catalog

The system maintains a catalog of *units*—words and multi-word phrases observed in a corpus. Each unit u stores:

- **Left context distribution** L_u : words appearing before u
- **Right context distribution** R_u : words appearing after u
- **Frequency count**: how often u appears

2.2 Bidirectional Parsing

Given an input sequence, the parser constructs a hierarchical parse tree:

1. Each token is initially a leaf node
2. Adjacent spans are merged based on an *energy function* that scores how well the combined span matches a unit in the catalog
3. The process continues until a complete tree is formed
4. The parse with minimum total energy is selected

The energy function combines:

$$E(\text{span}) = -\log P(\text{unit}) - \lambda \cdot \text{context_match} \quad (1)$$

2.3 Substitution Class Computation

For a unit u , its substitution class contains other units with similar context patterns:

$$\text{SubClass}(u) = \{v : \text{sim}(L_u, L_v) + \text{sim}(R_u, R_v) > \theta\} \quad (2)$$

where sim is a similarity measure (cosine or Lin’s measure, discussed below).

2.4 Prediction via Class Expansion

To predict continuations after a parsed input:

Algorithm: Prediction with Substitution Class Expansion

1. Parse input into hierarchy T
2. Initialize predictions $\leftarrow \{\}$
3. For each node n in T :
 - (a) Compute class $\leftarrow \text{SubClass}(n.\text{unit})$
 - (b) For each member m in class:

- i. For each word w in m 's right context:
 $\text{predictions}[w] += \text{centrality}(m) \cdot P(w|m)$
4. Return top-k predictions

The key insight is that predictions aggregate from *all hierarchy levels* and *all class members*, allowing abstract representations to contribute alongside specific ones.

3 Lin's Information-Theoretic Similarity

3.1 Motivation

The original cosine similarity measure sometimes included structurally dissimilar units in substitution classes. For example, "we should go" might include "prefer" because both appear in conversational contexts ("I think...", "maybe...") even though they have different grammatical structures.

3.2 Lin's Measure

We implemented Lin's information-theoretic similarity [1]:

$$\text{sim}(A, B) = \frac{2 \cdot I(\text{common}(A, B))}{I(\text{desc}(A)) + I(\text{desc}(B))} \quad (3)$$

where:

- $I(x) = -\log P(x)$ is the information content
- $\text{common}(A, B)$ is the set of context words shared by both A and B
- $\text{desc}(A)$ is the full description (all context words) of A

3.3 Key Property

Lin's measure weights *rare* shared features more heavily than common ones. If two units share the rare context word "meteorological", this contributes more to their similarity than sharing "the".

This naturally filters out units that share only common conversational markers, retaining those with genuine structural similarity.

3.4 Results

With Lin similarity, the substitution class for "we should go" now contains:

- "should go" (0.678)
- "you'd better" (0.398)
- "you have to go" (0.350)

Previously problematic matches like "prefer" and "there's a place" are filtered out.

4 GPU Optimizations

4.1 Challenge

Computing substitution classes requires comparing a query unit against thousands of candidates. With 70,000 units and vocabulary size 47,000, this was prohibitively slow on CPU.

4.2 Implementation

We implemented GPU-accelerated similarity computation using PyTorch with Apple Metal (MPS) support:

1. **Pre-compute dense matrices:** Store context distributions as GPU tensors
 - Binary matrices for Lin similarity: $(n_units \times vocab_size)$
 - Information content vector: $(vocab_size,)$
 - Pre-computed $I(desc)$ for each unit: $(n_units,)$
2. **Batch similarity computation:** Compute all candidate similarities in parallel

```
# Common features via element-wise multiplication
common = candidates_binary * query_binary # (n_cands, vocab)

# I(common) via matrix-vector multiply
I_common = common @ info_content # (n_cands,)

# Lin's formula
similarity = 2 * I_common / (I_desc_query + I_desc_candidates)
```

3. **Memory optimization:** Lin similarity only needs binary matrices (presence/absence), not weighted matrices, reducing memory from 24GB to 12GB

4.3 Performance

Operation	Before	After
Substitution class lookup	60+ seconds	<1 second
Parser span processing	17-20x slower	Baseline
Memory usage (Lin)	N/A	12.4 GB

5 Current Architecture

5.1 Files

- `bidir_simple.py`: Core parser, unit catalog, context patterns
- `substitution_predictor.py`: Prediction engine with GPU acceleration
- `analyze_phrase.py`: User-facing tool for phrase analysis

5.2 Usage

```
python analyze_phrase.py "we_should_go"
```

Output includes:

1. Parse tree of the input
2. Substitution classes for each sub-span
3. Predicted continuations

6 Future Directions

1. **Prediction quality:** Current predictions aggregate from substitution classes but may still include semantically odd continuations. Further tuning of aggregation weights and thresholds is needed.
2. **Sparse unit handling:** Units with low corpus frequency have sparse context distributions, making Lin similarity less effective. Fallback strategies or smoothing techniques may help.
3. **Benchmark comparisons:** Systematic comparison with transformer language models on standard benchmarks would quantify the approach’s strengths and weaknesses.
4. **Incremental updates:** Currently the catalog is static. Supporting incremental updates as new text is observed would enable online learning.

7 Conclusion

Substitution class expansion provides a mechanism analogous to transformer attention: both relate inputs to contextually equivalent forms and aggregate predictions from this broader set. The key difference is that substitution classes are computed dynamically from corpus statistics rather than learned from gradient descent.

Lin’s information-theoretic similarity improves structural filtering by weighting rare shared features, and GPU acceleration makes the approach practical for real-time use.

References

- [1] Lin, D. (1998). An information-theoretic definition of similarity. In *Proceedings of the 15th International Conference on Machine Learning* (pp. 296-304).